

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ХАРКІВСЬКИЙ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ

з лабораторної роботи №3

з дисципліни "Основи програмної інженерії"

Виконала

ст. гр. ПЗПІ-25-5

Жакун А. О

Прийняв

Гребенюк В. О

ЛАБОРАТОРНА РОБОТА 3

МОДУЛЬНЕ ТЕСТУВАННЯ ПРОГРАМНОГО КОДУ (UNIT TESTING)

3.1 Мета роботи

Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

Під час роботи було виконано бонусне завдання.

1. На основі UML-моделі з ЛР 02:

```
'use strict';
```

```
//
```

```
// Моделі даних (відповідають UML class diagram)
```

```
//
```

```
/**
```

```
 * Команда з рейтингом.
```

```
 * @typedef {Object} Team
```

```
 * @property {number} teamId
```

```
 * @property {string} name
```

```
 * @property {number} rating – поточний рейтинг [0..100]
```

```
 * @property {number} wins
```

```
 * @property {number} losses
```

```
 * @property {number} draws
```

```
 * @property {number} goalsScored
```

```
 * @property {number} goalsConceded
```

```
 * @property {string[]} last5Matches – масив результатів: 'W' | 'D' | 'L'
```

```
 */
```

```
/**
```

```
 * Статистика одного матчу.
```

```
 * @typedef {Object} MatchStats
```

```
 * @property {number} possessionHome
```

```
 * @property {number} possessionAway
```

```
 * @property {number} shotsHome
```

```
 * @property {number} shotsAway
```

```
 * @property {number} foulsHome
```

```
* @property {number} foulsAway
*/
```

```
/**
```

```
* Матч між двома командами.
* @typedef {Object} Match
* @property {number} matchId
* @property {string} date – ISO-рядок дати, напр. '2024-05-01'
* @property {Team} teamHome
* @property {Team} teamAway
* @property {number} scoreHome
* @property {number} scoreAway
* @property {string} sport – вид спорту, напр. 'football'
* @property {MatchStats|null} stats
*/
```

```
/**
```

```
* Персоналізована рекомендація.
* @typedef {Object} Recommendation
* @property {number} recId
* @property {number} matchId
* @property {number} odds
* @property {number} confidence – [0..1]
* @property {string} description
*/
```

```
/**
```

```
* Користувач системи.
* @typedef {Object} User
* @property {number} userId
* @property {string} email
* @property {string[]} preferredSports
*/
```

```
//
```

```
// Фабричні функції (замість конструкторів)
//
```

```
/**
```

```
* Створює об'єкт Team.
* @param {number} teamId
* @param {string} name
* @param {Object} [stats={}]
* @returns {Team}
*/
```

```
function createTeam(teamId, name, stats = {}) {
  if (!Number.isInteger(teamId) || teamId <= 0) {
```

```

        throw new Error('teamId must be a positive integer');
    }
    if (typeof name !== 'string' || name.trim() === '') {
        throw new Error('name must be a non-empty string');
    }

    return {
        teamId,
        name: name.trim(),
        wins:      stats.wins      ?? 0,
        losses:    stats.losses    ?? 0,
        draws:     stats.draws     ?? 0,
        goalsScored: stats.goalsScored ?? 0,
        goalsConceded: stats.goalsConceded ?? 0,
        last5Matches: stats.last5Matches ?? [],
        rating:     stats.rating     ?? 0,
    };
}

/**
 * Создание об'екта Match.
 * @param {number}    matchId
 * @param {string}    date
 * @param {Team}      teamHome
 * @param {Team}      teamAway
 * @param {Object}    [opts={}]
 * @returns {Match}
 */
function createMatch(matchId, date, teamHome, teamAway, opts = {}) {
    if (!Number.isInteger(matchId) || matchId <= 0) {
        throw new Error('matchId must be a positive integer');
    }
    if (typeof date !== 'string' || date.trim() === '') {
        throw new Error('date must be a non-empty string');
    }
    if (!teamHome || !teamAway) {
        throw new Error('teamHome and teamAway are required');
    }

    return {
        matchId,
        date: date.trim(),
        teamHome,
        teamAway,
        scoreHome: opts.scoreHome ?? 0,
        scoreAway: opts.scoreAway ?? 0,
        sport:     opts.sport      ?? 'football',
        stats:     opts.stats      ?? null,
    };
}

```

```
/**
 * Створює об'єкт User.
 * @param {number} userId
 * @param {string} email
 * @param {string[]} [preferredSports=[]]
 * @returns {User}
 */
function createUser(userId, email, preferredSports = []) {
  if (!Number.isInteger(userId) || userId <= 0) {
    throw new Error('userId must be a positive integer');
  }
  if (typeof email !== 'string' || !email.includes('@')) {
    throw new Error('email must be a valid email address');
  }

  return {
    userId,
    email: email.trim(),
    preferredSports: Array.isArray(preferredSports) ? [...preferredSports] : [],
  };
}
```

```
//
```

```
// KGLG-03 — StatsService
//
```

```
/**
 * Сервіс статистики матчів та рейтингів команд.
 *
 * Зберігає список матчів у пам'яті (імітація бази даних / API-кешу).
 * Відповідає вимозі KGLG-03.
 */
class StatsService {
  /**
   * @param {Match[]} [matches=[]]
   */
  constructor(matches = []) {
    if (!Array.isArray(matches)) {
      throw new Error('matches must be an array');
    }
    this._matches = [...matches];
  }

  // — публічний API
```

```
/**
```

```

* Повертає копію повного списку матчів.
* @returns {Match[]}
*/
getMatchList() {
    return [...this._matches];
}

/**
* Повертає статистику для матчу з вказаним matchId.
*
* @param {number} matchId
* @returns {MatchStats}
* @throws {Error} якщо matchId не ціле число, матч не знайдено
*             або статистика ще не завантажена.
*/
getMatchStats(matchId) {
    if (!Number.isInteger(matchId)) {
        throw new Error('matchId must be an integer');
    }

    const match = this._findMatch(matchId);

    if (!match.stats) {
        throw new Error(`Stats for match ${matchId} are not loaded yet`);
    }

    return match.stats;
}

/**
* Знаходить і повертає рейтинг команди за teamId серед усіх матчів.
*
* @param {number} teamId
* @returns {number}
* @throws {Error} якщо teamId не ціле число або команду не знайдено.
*/
getTeamRating(teamId) {
    if (!Number.isInteger(teamId)) {
        throw new Error('teamId must be an integer');
    }

    for (const match of this._matches) {
        if (match.teamHome.teamId === teamId) return match.teamHome.rating;
        if (match.teamAway.teamId === teamId) return match.teamAway.rating;
    }

    throw new Error(`Team with id=${teamId} not found`);
}

/**
* Обчислює рейтинг команди за формулою з LogicComand.js (ЛР 01).

```

```

*
* Формула:
*  $winRate = (wins / totalMatches) * 100$ 
*  $goalFactor = (goalsScored - goalsConceded) * 0.2$ 
*  $formFactor = last5Score * 0.5 \quad (W=3, D=1, L=0)$ 
*  $rating = clamp(winRate + goalFactor + formFactor, 0, 100)$ 
*
* @param {Team} team
* @returns {number} – рейтинг у [0..100]
* @throws {Error} якщо team не є об'єктом або містить некоректні дані.
*/
calculateTeamRating(team) {
  if (!team || typeof team !== 'object') {
    throw new Error('team must be a non-null object');
  }

  const totalMatches = (team.wins ?? 0) + (team.losses ?? 0) + (team.draws ?? 0);

  if (totalMatches === 0) return 0;

  const winRate = (team.wins / totalMatches) * 100;
  const goalFactor = ((team.goalsScored ?? 0) - (team.goalsConceded ?? 0)) * 0.2;
  const formFactor = this._calcFormFactor(team.last5Matches ?? []);

  const rating = winRate + goalFactor + formFactor;
  return Math.round(Math.max(0, Math.min(100, rating)) * 100) / 100;
}

/**
* Фільтрує матчі за рядковим критерієм.
*
* Підтримувані критерії:
* 'sport:<назва>' — матчі вказаного виду спорту
* 'home_win' — перемога господарів
* 'away_win' — перемога гостей
* 'draw' — нічия
* 'high_rated' — обидві команди мають рейтинг ≥ 70
*
* @param {string} criteria
* @returns {Match[]}
* @throws {Error} якщо criteria порожній або невідомий.
*/
filterStats(criteria) {
  if (typeof criteria !== 'string' || criteria.trim() === '') {
    throw new Error('criteria must be a non-empty string');
  }

  const c = criteria.trim().toLowerCase();

  if (c.startsWith('sport:')) {
    const sportName = c.slice('sport:'.length).trim();

```

```

    if (!sportName) {
        throw new Error("sport name cannot be empty in 'sport:' criteria");
    }
    return this._matches.filter(m => m.sport.toLowerCase() === sportName);
}

if (c === 'home_win') {
    return this._matches.filter(m => m.scoreHome > m.scoreAway);
}

if (c === 'away_win') {
    return this._matches.filter(m => m.scoreAway > m.scoreHome);
}

if (c === 'draw') {
    return this._matches.filter(m => m.scoreHome === m.scoreAway);
}

if (c === 'high_rated') {
    return this._matches.filter(
        m => m.teamHome.rating >= 70 && m.teamAway.rating >= 70
    );
}

throw new Error(`Unknown filter criteria: '${criteria}'`);
}

```

// — внутрішні методи

```

/**
 * @param {number} matchId
 * @returns {Match}
 * @private
 */
_findMatch(matchId) {
    const match = this._matches.find(m => m.matchId === matchId);
    if (!match) {
        throw new Error(`Match with id=${matchId} not found`);
    }
    return match;
}

/**
 * Обчислює formFactor з масиву результатів останніх 5 матчів.
 *  $W \rightarrow 3$  бали,  $D \rightarrow 1$ ,  $L \rightarrow 0$ .
 *
 * @param {string[]} last5
 * @returns {number}
 * @private
 */

```



```
_calcFormFactor(last5) {  
  const scoreMap = { W: 3, D: 1, L: 0 };  
  const rawScore = last5.reduce((sum, r) => sum + (scoreMap[r] ?? 0), 0);  
  return rawScore * 0.5;  
}  
}
```

```
//
```

```
// KGLG-04 — RecommendationEngine
```

```
//
```

```
/**
```

```
 * Рушій персоналізованих рекомендацій матчів.
```

```
 *
```

```
 * Рекомендує матчі, вид спорту яких збігається з уподобаннями
```

```
 * користувача; впевненість обчислюється на основі різниці рейтингів.
```

```
 * Відповідає вимозі KGLG-04.
```

```
 */
```

```
class RecommendationEngine {
```

```
  /** Порог впевненості — рекомендації нижче порогу відкидаються. */
```

```
  static CONFIDENCE_THRESHOLD = 0.1;
```

```
  /**
```

```
   * @param {Match[]} [matches=[]]
```

```
   */
```

```
  constructor(matches = []) {
```

```
    if (!Array.isArray(matches)) {
```

```
      throw new Error('matches must be an array');
```

```
    }
```

```
    this._matches = [...matches];
```

```
    this._recCounter = 0;
```

```
  }
```

```
  // — публічний API
```

```
  /**
```

```
   * Генерує відсортований за впевненістю список рекомендацій для user.
```

```
   *
```

```
   * Алгоритм (з sequence diagram fr-04-sd):
```

```
   * 1. Визначити уподобання користувача (preferredSports).
```

```
   * 2. Для кожного матчу, вид спорту якого є в уподобаннях:
```

```
   *   a. Обчислити впевненість.
```

```
   *   b. Якщо  $confidence \geq CONFIDENCE\_THRESHOLD \rightarrow$  додати рекомендацію.
```

```
   * 3. Повернути список, відсортований від вищої впевненості до нижчої.
```

```
   *
```

```
   * @param {User} user
```

```

* @returns {Recommendation[]}
* @throws {Error} якщо user не містить поле preferredSports
*      або список уподобань порожній.
*/
generateRecommendations(user) {
  if (!user || typeof user !== 'object') {
    throw new Error('user must be a non-null object');
  }
  if (!Array.isArray(user.preferredSports)) {
    throw new Error('user.preferredSports must be an array');
  }
  if (user.preferredSports.length === 0) {
    throw new Error(`User ${user.userId} has no preferred sports set`);
  }

  const sportsSet = new Set(
    user.preferredSports.map(s => s.toLowerCase().trim())
  );

  const recommendations = [];

  for (const match of this._matches) {
    if (!sportsSet.has(match.sport.toLowerCase())) continue;

    const confidence = this.calculateConfidence(match);
    if (confidence < RecommendationEngine.CONFIDENCE_THRESHOLD) continue;

    const odds      = this._estimateOdds(confidence);
    const description = this._buildDescription(match, confidence);

    this._recCounter += 1;
    recommendations.push({
      recId:    this._recCounter,
      matchId:  match.matchId,
      odds,
      confidence,
      description,
    });
  }

  recommendations.sort((a, b) => b.confidence - a.confidence);
  return recommendations;
}

/**
 * Повертає нормалізований (lower-case) список уподобань користувача.
 *
 * @param {User} user
 * @returns {string[]}
 * @throws {Error} якщо user не містить коректного поля preferredSports.
 */

```

```

analyzePreferences(user) {
  if (!user || typeof user !== 'object') {
    throw new Error('user must be a non-null object');
  }
  if (!Array.isArray(user.preferredSports)) {
    throw new Error('user.preferredSports must be an array');
  }

  return user.preferredSports.map(s => s.toLowerCase().trim());
}

/**
 * Обчислює впевненість прогнозу для матчу на основі різниці рейтингів.
 *
 * Формула:
 *  $confidence = |ratingHome - ratingAway| / 100$ 
 *
 * Результат обмежений діапазоном [0..1].
 *
 * @param {Match} match
 * @returns {number}
 * @throws {Error} якщо match не є об'єктом або не має команд.
 */
calculateConfidence(match) {
  if (!match || typeof match !== 'object') {
    throw new Error('match must be a non-null object');
  }
  if (!match.teamHome || !match.teamAway) {
    throw new Error('match must have teamHome and teamAway');
  }

  const diff = Math.abs(match.teamHome.rating - match.teamAway.rating);
  return Math.round(Math.min(diff / 100, 1) * 10000) / 10000;
}

// — внутрішні методи

```

```

/**
 * Оцінює коефіцієнт: чим вища впевненість, тим нижчий коефіцієнт.
 * @param {number} confidence
 * @returns {number}
 * @private
 */
_estimateOdds(confidence) {
  if (confidence <= 0) return 3.0;
  const odds = 1 + (1 - confidence) * 2;
  return Math.round(Math.max(1.01, odds) * 100) / 100;
}

/**

```

```
* Формує текстовий опис рекомендації.
* @param {Match} match
* @param {number} confidence
* @returns {string}
* @private
*/
_buildDescription(match, confidence) {
  const favourite =
    match.teamHome.rating >= match.teamAway.rating
      ? match.teamHome.name
      : match.teamAway.name;

  const pct = (confidence * 100).toFixed(1);
  return (
    `${match.teamHome.name} vs ${match.teamAway.name} ` +
    `(${match.sport}): прогноз на перемогу ${favourite}, ` +
    `впевненість ${pct}%`
  );
}
}

//



---




---


// Експорт
//



---




---



module.exports = {
  createTeam,
  createMatch,
```

2. Класи еквівалентності (EP) та граничні значення (BVA)

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
СТ-01	teamId=1, name='Dynamo'	Team з teamId=1, name='Dynamo'	EP	pass
СТ-02	teamId=1 (мін. допустиме)	Team успішно створено	BVA	pass
СТ-03	teamId=0 (межа — нуль)	throw Error 'positive integer'	BVA	fail
СТ-04	teamId=-1 (від'ємне)	throw Error 'positive integer'	EP	fail
СТ-05	teamId=1.5 (не ціле)	throw Error 'positive integer'	EP	fail
СТ-06	name="" (порожній рядок)	throw Error 'non-empty string'	BVA	fail
СТ-07	name=' Kyiv ' (з пробілами)	team.name='Kyiv' (trim)	EP	pass
СТ-08	stats не передано	wins=0, losses=0, rating=0	EP	pass

```

createUser,
StatsService,
RecommendationEngine,
};

```

```

createTeam(teamId, name, stats)
StatsService.getMatchStats(matchId)

```

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
GS-01	matchId=1, stats завантажені	об'єкт MatchStats	EP	pass
GS-02	matchId=1, stats=null	throw Error 'not loaded yet'	EP	fail
GS-03	matchId=999 (не існує)	throw Error 'not found'	EP	fail
GS-04	matchId='abc' (не число)	throw Error 'must be integer'	EP	fail
GS-05	matches=[], matchId=1	throw Error 'not found'	BVA	fail

StatsService.getTeamRating(teamId)

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
GTR-01	teamId=1 (є в teamHome)	rating господарської команди	EP	pass
GTR-02	teamId=2 (є в teamAway)	rating гостьової команди	EP	pass
GTR-03	teamId=999 (не існує)	throw Error 'not found'	EP	fail
GTR-04	teamId=1.5 (не ціле)	throw Error 'must be integer'	EP	fail

StatsService.calculateTeamRating(team)

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
CTR-01	wins=0, losses=0, draws=0	0 (totalMatches=0, межа)	BVA	pass
CTR-02	wins=1, losses=0, draws=0	100.0 (totalMatches=1, межа+1)	BVA	pass
CTR-03	wins=10, losses=5, draws=5, goals +5	число у діапазоні [0..100]	EP	pass
CTR-04	wins=30, goals +100 (вихід за 100)	100.0 (clamp до максимуму)	BVA	pass
CTR-05	wins=0, goals -200 (вихід за 0)	0 (clamp до мінімуму)	BVA	pass
CTR-06	last5=['W','W','W','W','W']	formFactor=7.5	EP	pass
CTR-07	team=null	throw Error 'non-null object'	EP	fail

StatsService.filterStats(criteria)

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
FS-01	'home_win', матч 2:1	масив з 1 матчу	EP	pass
FS-02	'away_win', матч 0:3	масив з 1 матчу	EP	pass
FS-03	'draw', матч 1:1	масив з 1 матчу	EP	pass

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
FS-04	'sport:football'	тільки матчі з sport='football'	EP	pass
FS-05	'highRated', rating=80 та 75	масив з 1 матчу	EP	pass
FS-06	'highRated', rating=70 та 70 (межа)	масив з 1 матчу (включно)	BVA	pass
FS-07	'highRated', rating=69 та 80 (межа-1)	порожній масив []	BVA	pass
FS-08	criteria="" (порожній рядок)	throw Error 'non-empty string'	BVA	fail
FS-09	criteria='unknown_filter'	throw Error 'Unknown criteria'	EP	fail
FS-10	criteria='sport:' (без назви)	throw Error 'cannot be empty'	BVA	fail

RecommendationEngine.calculateConfidence(match)

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
CC-01	rating=80 та 80 (diff=0)	0.0	BVA	pass
CC-02	rating=80 та 70 (diff=10, поріг)	0.1	BVA	pass
CC-03	rating=80 та 65 (diff=15)	0.15	EP	pass

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
CC-04	rating=100 та 0 (diff=100, max)	1.0 (clamp)	BVA	pass
CC-05	match=null	throw Error 'non-null object'	EP	fail
CC-06	match без полів teamHome/teamAway	throw Error 'teamHome and teamAway'	EP	fail

RecommendationEngine.analyzePreferences(user)

Тест - кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
AP-01	preferredSports=['Football','Basketball']	['football','basketball']	EP	pass
AP-02	preferredSports=[' Tennis ']	['tennis'] (trim)	EP	pass
AP-03	preferredSports=[] (0 елементів)	[] (порожній масив)	BVA	pass
AP-04	user=null	throw Error 'non-null object'	EP	fail
AP-05	preferredSports='football' (рядок)	throw Error 'must be an array'	EP	fail

RecommendationEngine.generateRecommendations(user)

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
GRec-01	user=['football'], match.sport='football', diff=15	масив з 1 рекомендації	EP	pass
GRec-02	user=['basketball'], match.sport='football'	порожній масив []	EP	pass
GRec-03	diff рейтингів=10 (confidence=0.1, рівно поріг)	рекомендацію включено	BVA	pass
GRec-04	diff рейтингів=9 (confidence=0.09, нижче порогу)	рекомендацію виключено, []	BVA	pass
GRec-05	3 матчі з diff 30, 15, 20	відсортовано: 0.3, 0.2, 0.15	EP	pass
GRec-06	preferredSports=[] (порожній)	throw Error 'no preferred sports'	BVA	fail
GRec-07	user=null	throw Error 'non-null object'	EP	fail
GRec-08	matches=[], user з уподобаннями	порожній масив []	BVA	pass
GRec-09	user=['Football'], match.sport='football'	1 рекомендація (case-insensitive)	EP	pass

3. Вихідний код тестового набору з коментарями:

```

4. 'use strict';
5.
6. const {
7.   createTeam,
8.   createMatch,
9.   createUser,

```

```

10. StatsService,
11. RecommendationEngine,
12. } = require('./2Win');
13.
14.//

```

```

15.// ДОПОМІЖНІ ФАБРИКИ (спрощують створення тестових даних)
16.//

```

```

17.
18./** Створює команду з рейтингом без зайвих параметрів */
19.function makeTeam(id, name, rating = 50, extra = {}) {
20.  return createTeam(id, name, { rating, ...extra });
21.}
22.
23./** Створює матч із двома командами */
24.function makeMatch(id, home, away, opts = {}) {
25.  return createMatch(id, '2024-05-01', home, away, opts);
26.}
27.
28.//

```

```

29.// 1. createTeam
30.//

```

```

31.
32.describe('createTeam', () => {
33.  // СТ-01 – позитивний / EP: допустимий клас вхідних даних
34.  test('СТ-01 [EP+] повертає команду з коректними teamId та name', () => {
35.    // Arrange
36.    const teamId = 1;
37.    const name = 'Dynamo';
38.
39.    // Act
40.    const team = createTeam(teamId, name);
41.
42.    // Assert
43.    expect(team.teamId).toBe(1);
44.    expect(team.name).toBe('Dynamo');
45.  });
46.
47.  // СТ-02 – позитивний / BVA: мінімально допустиме значення teamId = 1
48.  test('СТ-02 [BVA+] teamId=1 (мінімальна межа) – успішне створення', () => {
49.    // Arrange / Act
50.    const team = createTeam(1, 'Shakhtar');
51.
52.    // Assert
53.    expect(team.teamId).toBe(1);
54.  });
55.
56.  // СТ-03 – негативний / BVA: teamId=0 (межа – нуль, нижче мінімуму)
57.  test('СТ-03 [BVA-] teamId=0 кидає помилку "positive integer"', () => {
58.    // Arrange

```

```
59.     const invalidId = 0;
60.
61.     // Act & Assert
62.     expect(() => createTeam(invalidId, 'Test')).toThrow('positive integer');
63. });
64.
65. // CT-04 – негативний / EP: від'ємний teamId
66. test('CT-04 [EP-] teamId=-1 кидає помилку "positive integer"', () => {
67.     // Arrange
68.     const invalidId = -1;
69.
70.     // Act & Assert
71.     expect(() => createTeam(invalidId, 'Test')).toThrow('positive integer');
72. });
73.
74. // CT-05 – негативний / EP: не ціле число
75. test('CT-05 [EP-] teamId=1.5 (дробове) кидає помилку "positive integer"',
    () => {
76.     // Arrange
77.     const invalidId = 1.5;
78.
79.     // Act & Assert
80.     expect(() => createTeam(invalidId, 'Test')).toThrow('positive integer');
81. });
82.
83. // CT-06 – негативний / BVA: name = '' (порожній рядок – межа)
84. test('CT-06 [BVA-] name="" (порожній рядок) кидає помилку "non-empty
    string"', () => {
85.     // Arrange
86.     const emptyName = '';
87.
88.     // Act & Assert
89.     expect(() => createTeam(1, emptyName)).toThrow('non-empty string');
90. });
91.
92. // CT-07 – позитивний / EP: name з пробілами → trim()
93. test('CT-07 [EP+] name=" Kyiv " обрізається до "Kyiv"', () => {
94.     // Arrange
95.     const paddedName = ' Kyiv ';
96.
97.     // Act
98.     const team = createTeam(1, paddedName);
99.
100.    // Assert
101.    expect(team.name).toBe('Kyiv');
102. });
103.
104. // CT-08 – позитивний / EP: stats не передається → усі значення = 0
105. test('CT-08 [EP+] без stats всі числові поля = 0', () => {
106.     // Arrange / Act
107.     const team = createTeam(1, 'Default');
108.
109.     // Assert
```

```
110.         expect(team.wins).toBe(0);
111.         expect(team.losses).toBe(0);
112.         expect(team.rating).toBe(0);
113.     });
114. });
115.
116. //

```

```
117. // 2. StatsService.getMatchStats
118. //

```

```
119.
120. describe('StatsService.getMatchStats', () => {
121.     // GS-01 – позитивний / EP: матч зі stats → повертає об'єкт
    MatchStats
122.     test('GS-01 [EP+] matchId існує і stats завантажені → повертає
    stats', () => {
123.         // Arrange
124.         const statsData = { possessionHome: 55, possessionAway: 45,
            shotsHome: 10, shotsAway: 7, foulsHome: 3, foulsAway: 4 };
125.         const home = makeTeam(1, 'Home');
126.         const away = makeTeam(2, 'Away');
127.         const match = makeMatch(1, home, away, { stats: statsData });
128.         const svc = new StatsService([match]);
129.
130.         // Act
131.         const result = svc.getMatchStats(1);
132.
133.         // Assert
134.         expect(result).toEqual(statsData);
135.     });
136.
137.     // GS-02 – негативний / EP: stats=null → виняток 'not loaded yet'
138.     test('GS-02 [EP-] stats=null кидає помилку "not loaded yet"', () => {
139.         // Arrange
140.         const home = makeTeam(1, 'Home');
141.         const away = makeTeam(2, 'Away');
142.         const match = makeMatch(1, home, away); // stats: null за
            замовчуванням
143.         const svc = new StatsService([match]);
144.
145.         // Act & Assert
146.         expect(() => svc.getMatchStats(1)).toThrow('not loaded yet');
147.     });
148.
149.     // GS-03 – негативний / EP: matchId не існує → виняток 'not found'
150.     test('GS-03 [EP-] matchId=999 не існує → кидає помилку "not found"',
        () => {
151.         // Arrange
152.         const svc = new StatsService([]);
153.
154.         // Act & Assert
155.         expect(() => svc.getMatchStats(999)).toThrow('not found');
```

```

156.     });
157.
158.     // GS-04 – негативний / EP: matchId не число → виняток 'must be
integer'
159.     test('GS-04 [EP-] matchId="abc" кидає помилку "must be integer"', ()
=> {
160.         // Arrange
161.         const svc = new StatsService([]);
162.
163.         // Act & Assert
164.         expect(() => svc.getMatchStats('abc')).toThrow('must be an
integer');
165.     });
166.
167.     // GS-05 – негативний / BVA: порожній список матчів + будь-який
matchId → 'not found'
168.     test('GS-05 [BVA-] matches=[] (порожній список) → кидає помилку "not
found"', () => {
169.         // Arrange
170.         const svc = new StatsService([]);
171.
172.         // Act & Assert
173.         expect(() => svc.getMatchStats(1)).toThrow('not found');
174.     });
175. });
176.
177. //

```

```

178. // 3. StatsService.getTeamRating
179. //

```

```

180.
181. describe('StatsService.getTeamRating', () => {
182.     let svc;
183.
184.     beforeEach(() => {
185.         const home = makeTeam(1, 'HomeTeam', 80);
186.         const away = makeTeam(2, 'AwayTeam', 65);
187.         const match = makeMatch(1, home, away);
188.         svc = new StatsService([match]);
189.     });
190.
191.     // GTR-01 – позитивний / EP: teamId є в teamHome → повертає rating
господарів
192.     test('GTR-01 [EP+] teamId=1 (teamHome) → повертає рейтинг 80', () =>
{
193.         // Arrange – виконано в beforeEach
194.         // Act
195.         const rating = svc.getTeamRating(1);
196.
197.         // Assert
198.         expect(rating).toBe(80);
199.     });

```

```

200.
201.     // GTR-02 – позитивний / EP: teamId є в teamAway → повертає rating
    зостей
202.     test('GTR-02 [EP+] teamId=2 (teamAway) → повертає рейтинг 65', () =>
    {
203.         // Arrange – виконано в beforeEach
204.         // Act
205.         const rating = svc.getTeamRating(2);
206.
207.         // Assert
208.         expect(rating).toBe(65);
209.     });
210.
211.     // GTR-03 – негативний / EP: teamId не існує → виняток 'not found'
212.     test('GTR-03 [EP-] teamId=999 → кидає помилку "not found"', () => {
213.         // Act & Assert
214.         expect(() => svc.getTeamRating(999)).toThrow('not found');
215.     });
216.
217.     // GTR-04 – негативний / EP: teamId не ціле → виняток 'must be
    integer'
218.     test('GTR-04 [EP-] teamId=1.5 → кидає помилку "must be integer"', ()
    => {
219.         // Act & Assert
220.         expect(() => svc.getTeamRating(1.5)).toThrow('must be an integer');
221.     });
222. });
223.
224. //

```

```

225. // 4. StatsService.calculateTeamRating
226. //

```

```

227.
228. describe('StatsService.calculateTeamRating', () => {
229.     let svc;
230.
231.     beforeEach(() => {
232.         svc = new StatsService([]);
233.     });
234.
235.     // CTR-01 – позитивний / BVA: totalMatches=0 (межа нуль) → rating=0
236.     test('CTR-01 [BVA+] totalMatches=0 → повертає 0', () => {
237.         // Arrange
238.         const team = { wins: 0, losses: 0, draws: 0, goalsScored: 0,
            goalsConceded: 0, last5Matches: [] };
239.
240.         // Act
241.         const result = svc.calculateTeamRating(team);
242.
243.         // Assert
244.         expect(result).toBe(0);
245.     });

```

```
246.
247.     // CTR-02 – позитивний / BVA: totalMatches=1 (межа+1), 1 перемога →
    winRate=100
248.     test('CTR-02 [BVA+] wins=1, totalMatches=1 → winRate=100 (але clamp
    може знизити)', () => {
249.         // Arrange
250.         const team = { wins: 1, losses: 0, draws: 0, goalsScored: 0,
    goalsConceded: 0, last5Matches: [] };
251.
252.         // Act
253.         const result = svc.calculateTeamRating(team);
254.
255.         // Assert – без голів та форми: winRate=100, обмежено до 100
256.         expect(result).toBe(100);
257.     });
258.
259.     // CTR-03 – позитивний / EP: типовий розрахунок → результат в
    [0..100]
260.     test('CTR-03 [EP+] wins=10, losses=5, draws=5, goals+5 → рейтинг у
    [0..100]', () => {
261.         // Arrange
262.         const team = { wins: 10, losses: 5, draws: 5, goalsScored: 25,
    goalsConceded: 20, last5Matches: ['W','D','L','W','D'] };
263.
264.         // Act
265.         const result = svc.calculateTeamRating(team);
266.
267.         // Assert
268.         expect(result).toBeGreaterThanOrEqual(0);
269.         expect(result).toBeLessThanOrEqual(100);
270.     });
271.
272.     // CTR-04 – позитивний / BVA: дуже висока сума → clamp до 100
273.     test('CTR-04 [BVA+] wins=30, goals+100 → clamp до 100', () => {
274.         // Arrange
275.         const team = { wins: 30, losses: 0, draws: 0, goalsScored: 100,
    goalsConceded: 0, last5Matches: ['W','W','W','W','W'] };
276.
277.         // Act
278.         const result = svc.calculateTeamRating(team);
279.
280.         // Assert
281.         expect(result).toBe(100);
282.     });
283.
284.     // CTR-05 – позитивний / BVA: від'ємний goalFactor → clamp до 0
285.     test('CTR-05 [BVA+] wins=0, goals-200 → clamp до 0', () => {
286.         // Arrange
287.         const team = { wins: 0, losses: 10, draws: 0, goalsScored: 0,
    goalsConceded: 200, last5Matches: [] };
288.
289.         // Act
290.         const result = svc.calculateTeamRating(team);
```



```

291.
292.     // Assert
293.     expect(result).toBe(0);
294. });
295.
296.     // CTR-06 – позитивний / EP: 5 перемог в last5 → formFactor = 7.5
297.     test('CTR-06 [EP+] last5=["W","W","W","W","W"] → formFactor=7.5', ()
    => {
298.         // Arrange – команда без перемог, щоб ізолювати formFactor
299.         const team = {
300.             wins: 0, losses: 10, draws: 0,
301.             goalsScored: 0, goalsConceded: 0,
302.             last5Matches: ['W', 'W', 'W', 'W', 'W'],
303.         };
304.
305.         // Act
306.         const result = svc.calculateTeamRating(team);
307.         // winRate=0, goalFactor=0, formFactor=7.5 → rating=7.5
308.
309.         // Assert
310.         expect(result).toBeCloseTo(7.5, 5);
311.     });
312.
313.     // CTR-07 – негативний / EP: team=null → виняток 'non-null object'
314.     test('CTR-07 [EP-] team=null → кидає помилку "non-null object"', ()
    => {
315.         // Act & Assert
316.         expect(() => svc.calculateTeamRating(null)).toThrow('non-null
    object');
317.     });
318. });
319.
320. //

```

```

321.     // 5. StatsService.filterStats
322.     //

```

```

323.
324.     describe('StatsService.filterStats', () => {
325.         let svc;
326.         let matchHomeWin, matchAwayWin, matchDraw, matchHighRated,
            matchFootball, matchBasketball;
327.
328.         beforeEach(() => {
329.             const t1 = makeTeam(1, 'Alpha', 80);
330.             const t2 = makeTeam(2, 'Beta', 75);
331.             const t3 = makeTeam(3, 'Gamma', 69);
332.             const t4 = makeTeam(4, 'Delta', 70);
333.
334.             matchHomeWin = makeMatch(1, t1, t2, { scoreHome: 2, scoreAway:
                1, sport: 'football' });
335.             matchAwayWin = makeMatch(2, t1, t2, { scoreHome: 0, scoreAway:
                3, sport: 'football' });

```

```

336.         matchDraw          = makeMatch(3, t1, t2, { scoreHome: 1, scoreAway:
    1, sport: 'football' });
337.         matchHighRated     = makeMatch(4, t1, t2, { sport: 'football'
    });
    // rating 80+75 ≥ 70
338.         matchFootball      = makeMatch(5, t1, t2, { sport: 'football' });
339.         matchBasketball     = makeMatch(6, t1, t2, { sport: 'basketball' });
340.
341.         // Матч із командою нижче 70 для BVA
342.         const t5 = makeTeam(5, 'Low1', 69);
343.         const t6 = makeTeam(6, 'High1', 80);
344.         const matchBelowThreshold = makeMatch(7, t5, t6, { sport:
    'football' });
345.
346.         svc = new StatsService([
347.             matchHomeWin, matchAwayWin, matchDraw,
348.             matchHighRated, matchBasketball, matchBelowThreshold,
349.         ]);
350.     });
351.
352.     // FS-01 – позитивний / EP: home_win → лише матчі, де scoreHome >
    scoreAway
353.     test('FS-01 [EP+] "home_win" повертає матч із рахунком 2:1', () => {
354.         // Act
355.         const result = svc.filterStats('home_win');
356.
357.         // Assert
358.         expect(result).toHaveLength(1);
359.         expect(result[0].matchId).toBe(1);
360.     });
361.
362.     // FS-02 – позитивний / EP: away_win → лише матчі, де scoreAway >
    scoreHome
363.     test('FS-02 [EP+] "away_win" повертає матч із рахунком 0:3', () => {
364.         // Act
365.         const result = svc.filterStats('away_win');
366.
367.         // Assert
368.         expect(result).toHaveLength(1);
369.         expect(result[0].matchId).toBe(2);
370.     });
371.
372.     // FS-03 – позитивний / EP: draw → лише нічиї
373.     test('FS-03 [EP+] "draw" повертає матч із рахунком 1:1', () => {
374.         // Act
375.         const result = svc.filterStats('draw');
376.
377.         // Assert – матч matchId=3 (1:1) присутній у результаті; усі
    результати – нічиї
378.         expect(result.find(m => m.matchId === 3)).toBeDefined();
379.         result.forEach(m => expect(m.scoreHome).toBe(m.scoreAway));
380.     });
381.
382.     // FS-04 – позитивний / EP: sport:football → тільки матчі football

```

```

383.     test('FS-04 [EP+] "sport:football" повертає тільки футбольні матчі',
    () => {
384.         // Act
385.         const result = svc.filterStats('sport:football');
386.
387.         // Assert – усі матчі football (можемо всі, крім basketball)
388.         result.forEach(m => expect(m.sport).toBe('football'));
389.         const basketballCount = result.filter(m => m.sport ===
    'basketball').length;
390.         expect(basketballCount).toBe(0);
391.     });
392.
393.     // FS-05 – позитивний / EP: high_rated → обидві команди ≥ 70
394.     test('FS-05 [EP+] "high_rated" повертає матчі з обома командами
    рейтингу ≥ 70', () => {
395.         // Act
396.         const result = svc.filterStats('high_rated');
397.
398.         // Assert – у кожному матчі обидві команди ≥ 70
399.         result.forEach(m => {
400.             expect(m.teamHome.rating).toBeGreaterThanOrEqual(70);
401.             expect(m.teamAway.rating).toBeGreaterThanOrEqual(70);
402.         });
403.     });
404.
405.     // FS-06 – позитивний / BVA: high_rated з rating=70 та 70 (рівно
    межа) → включено
406.     test('FS-06 [BVA+] high_rated: rating=70 та 70 (рівно межа) →
    включено до результату', () => {
407.         // Arrange
408.         const t70a = makeTeam(10, 'T70A', 70);
409.         const t70b = makeTeam(11, 'T70B', 70);
410.         const matchBoundary = makeMatch(10, t70a, t70b);
411.         const svcBVA = new StatsService([matchBoundary]);
412.
413.         // Act
414.         const result = svcBVA.filterStats('high_rated');
415.
416.         // Assert
417.         expect(result).toHaveLength(1);
418.     });
419.
420.     // FS-07 – позитивний / BVA: high_rated з rating=69 та 80 (межа-1) →
    виключено
421.     test('FS-07 [BVA+] high_rated: rating=69 та 80 (нижче межі) →
    порожній масив', () => {
422.         // Arrange
423.         const tLow = makeTeam(20, 'LowTeam', 69);
424.         const tHigh = makeTeam(21, 'HighTeam', 80);
425.         const matchBelow = makeMatch(20, tLow, tHigh);
426.         const svcBVA = new StatsService([matchBelow]);
427.
428.         // Act

```

```

429.         const result = svcBVA.filterStats('high_rated');
430.
431.         // Assert
432.         expect(result).toHaveLength(0);
433.     });
434.
435.     // FS-08 – негативний / BVA: criteria='' (порожній рядок – межа) →
    виняток
436.     test('FS-08 [BVA-] criteria="" (порожній рядок) → кидає помилку "non-
    empty string"', () => {
437.         // Act & Assert
438.         expect(() => svc.filterStats('')).toThrow('non-empty string');
439.     });
440.
441.     // FS-09 – негативний / EP: невідомий критерій → виняток 'Unknown'
442.     test('FS-09 [EP-] criteria="unknown_filter" → кидає помилку
    "Unknown"', () => {
443.         // Act & Assert
444.         expect(() => svc.filterStats('unknown_filter')).toThrow('Unknown');
445.     });
446.
447.     // FS-10 – негативний / BVA: 'sport:' без назви → виняток 'cannot be
    empty'
448.     test('FS-10 [BVA-] criteria="sport:" (без назви спорту) → кидає
    помилку "cannot be empty"', () => {
449.         // Act & Assert
450.         expect(() => svc.filterStats('sport:')).toThrow('cannot be empty');
451.     });
452. });
453.
454. //

```

```

455. // 6. RecommendationEngine.calculateConfidence
456. //

```

```

457.
458. describe('RecommendationEngine.calculateConfidence', () => {
459.     let engine;
460.
461.     beforeEach(() => {
462.         engine = new RecommendationEngine([]);
463.     });
464.
465.     // CC-01 – позитивний / BVA: diff=0 (межа нуль) → confidence=0.0
466.     test('CC-01 [BVA+] rating=80 та 80 (diff=0) → confidence=0.0', () =>
    {
467.         // Arrange
468.         const match = makeMatch(1, makeTeam(1, 'H', 80), makeTeam(2, 'A',
    80));
469.
470.         // Act
471.         const result = engine.calculateConfidence(match);
472.

```

```

473.         // Assert
474.         expect(result).toBe(0.0);
475.     });
476.
477.     // CC-02 – позитивний / BVA: diff=10 (nopiz) → confidence=0.1
478.     test('CC-02 [BVA+] rating=80 та 70 (diff=10, nopir
CONFIDENCE_THRESHOLD) → 0.1', () => {
479.         // Arrange
480.         const match = makeMatch(1, makeTeam(1, 'H', 80), makeTeam(2, 'A',
70));
481.
482.         // Act
483.         const result = engine.calculateConfidence(match);
484.
485.         // Assert
486.         expect(result).toBeCloseTo(0.1, 4);
487.     });
488.
489.     // CC-03 – позитивний / EP: типова різниця → пропорційне значення
490.     test('CC-03 [EP+] rating=80 та 65 (diff=15) → confidence=0.15', () =>
{
491.         // Arrange
492.         const match = makeMatch(1, makeTeam(1, 'H', 80), makeTeam(2, 'A',
65));
493.
494.         // Act
495.         const result = engine.calculateConfidence(match);
496.
497.         // Assert
498.         expect(result).toBeCloseTo(0.15, 4);
499.     });
500.
501.     // CC-04 – позитивний / BVA: diff=100 (максимум) → clamp до 1.0
502.     test('CC-04 [BVA+] rating=100 та 0 (diff=100, максимум) → clamp до
1.0', () => {
503.         // Arrange
504.         const match = makeMatch(1, makeTeam(1, 'H', 100), makeTeam(2, 'A',
0));
505.
506.         // Act
507.         const result = engine.calculateConfidence(match);
508.
509.         // Assert
510.         expect(result).toBe(1.0);
511.     });
512.
513.     // CC-05 – негативний / EP: match=null → виняток 'non-null object'
514.     test('CC-05 [EP-] match=null → кидає помилку "non-null object"', ()
=> {
515.         // Act & Assert
516.         expect(() => engine.calculateConfidence(null)).toThrow('non-null
object');
517.     });

```

```

518.
519.     // CC-06 – негативний / EP: match без полів команд → виняток
    'teamHome and teamAway'
520.     test('CC-06 [EP-] match без teamHome/teamAway → кидает помилку', () =>
    {
521.         // Arrange
522.         const brokenMatch = { matchId: 1, sport: 'football' };
523.
524.         // Act & Assert
525.         expect(() =>
            engine.calculateConfidence(brokenMatch)).toThrow('teamHome and teamAway');
526.     });
527. });
528.
529. //

```

```

530. // 7. RecommendationEngine.analyzePreferences
531. //

```

```

532.
533. describe('RecommendationEngine.analyzePreferences', () => {
534.     let engine;
535.
536.     beforeEach(() => {
537.         engine = new RecommendationEngine([]);
538.     });
539.
540.     // AP-01 – позитивний / EP: звичайний список → нормалізований
    lowercase
541.     test('AP-01 [EP+] ["Football","Basketball"] →
    ["football","basketball"]', () => {
542.         // Arrange
543.         const user = createUser(1, 'user@test.com', ['Football',
    'Basketball']);
544.
545.         // Act
546.         const result = engine.analyzePreferences(user);
547.
548.         // Assert
549.         expect(result).toEqual(['football', 'basketball']);
550.     });
551.
552.     // AP-02 – позитивний / EP: рядок з пробілами → trim + lowercase
553.     test('AP-02 [EP+] [" Tennis "] → ["tennis"] (trim)', () => {
554.         // Arrange
555.         const user = createUser(1, 'user@test.com', [' Tennis ']);
556.
557.         // Act
558.         const result = engine.analyzePreferences(user);
559.
560.         // Assert
561.         expect(result).toEqual(['tennis']);
562.     });

```

```

563.
564.     // AP-03 – позитивний / BVA: порожній масив (межа 0 елементів) → []
565.     test('AP-03 [BVA+] preferredSports=[] → повертає порожній масив', ()
    => {
566.         // Arrange
567.         const user = createUser(1, 'user@test.com', []);
568.
569.         // Act
570.         const result = engine.analyzePreferences(user);
571.
572.         // Assert
573.         expect(result).toEqual([]);
574.     });
575.
576.     // AP-04 – негативний / EP: user=null → виняток 'non-null object'
577.     test('AP-04 [EP-] user=null → кидає помилку "non-null object"', () =>
    {
578.         // Act & Assert
579.         expect(() => engine.analyzePreferences(null)).toThrow('non-null
    object');
580.     });
581.
582.     // AP-05 – негативний / EP: preferredSports – рядок, а не масив →
    виняток
583.     test('AP-05 [EP-] preferredSports="football" (рядок замість масиву) →
    кидає помилку "must be an array"', () => {
584.         // Arrange
585.         const user = { userId: 1, email: 'u@test.com', preferredSports:
    'football' };
586.
587.         // Act & Assert
588.         expect(() => engine.analyzePreferences(user)).toThrow('must be an
    array');
589.     });
590. });
591.
592. //

```

```

593.     // 8. RecommendationEngine.generateRecommendations
594.     //

```

```

595.
596.     describe('RecommendationEngine.generateRecommendations', () => {
597.
598.         // GRec-01 – позитивний / EP: спорт збігається + confidence вище
    порогу → 1 рекомендація
599.         test('GRec-01 [EP+] football + diff=15 (0.15 ≥ 0.1) → 1
    рекомендація', () => {
600.             // Arrange
601.             const match = makeMatch(1, makeTeam(1, 'H', 80), makeTeam(2, 'A',
    65), { sport: 'football' });
602.             const engine = new RecommendationEngine([match]);
603.             const user = createUser(1, 'u@test.com', ['football']);

```

```

604.
605.         // Act
606.         const recs = engine.generateRecommendations(user);
607.
608.         // Assert
609.         expect(recs).toHaveLength(1);
610.         expect(recs[0].matchId).toBe(1);
611.     });
612.
613.     // GRec-02 – позитивний / EP: уподобання не збігаються зі спортом
        матчу → []
614.     test('GRec-02 [EP+] user=[basketball], match.sport=football →
        порожній масив', () => {
615.         // Arrange
616.         const match = makeMatch(1, makeTeam(1, 'H', 80), makeTeam(2, 'A',
        65), { sport: 'football' });
617.         const engine = new RecommendationEngine([match]);
618.         const user = createUser(1, 'u@test.com', ['basketball']);
619.
620.         // Act
621.         const recs = engine.generateRecommendations(user);
622.
623.         // Assert
624.         expect(recs).toHaveLength(0);
625.     });
626.
627.     // GRec-03 – позитивний / BVA: confidence=0.1 (рівно поріг) →
        рекомендацію включено
628.     test('GRec-03 [BVA+] diff=10 (confidence=0.1, рівно поріг) →
        рекомендацію включено', () => {
629.         // Arrange
630.         const match = makeMatch(1, makeTeam(1, 'H', 80), makeTeam(2, 'A',
        70), { sport: 'football' });
631.         const engine = new RecommendationEngine([match]);
632.         const user = createUser(1, 'u@test.com', ['football']);
633.
634.         // Act
635.         const recs = engine.generateRecommendations(user);
636.
637.         // Assert
638.         expect(recs).toHaveLength(1);
639.         expect(recs[0].confidence).toBeCloseTo(0.1, 4);
640.     });
641.
642.     // GRec-04 – позитивний / BVA: confidence=0.09 (нижче порогу) →
        рекомендацію виключено
643.     test('GRec-04 [BVA+] diff=9 (confidence=0.09, нижче порогу) →
        порожній масив', () => {
644.         // Arrange
645.         const match = makeMatch(1, makeTeam(1, 'H', 80), makeTeam(2, 'A',
        71), { sport: 'football' });
646.         const engine = new RecommendationEngine([match]);
647.         const user = createUser(1, 'u@test.com', ['football']);

```



```

648.
649.     // Act
650.     const recs = engine.generateRecommendations(user);
651.
652.     // Assert
653.     expect(recs).toHaveLength(0);
654. });
655.
656.     // GRec-05 – позитивний / EP: 3 матчі → відсортовано за confidence
        (спадання)
657.     test('GRec-05 [EP+] 3 матчі з diff 30,15,20 → відсортовано: 0.3, 0.2,
        0.15', () => {
658.         // Arrange
659.         const m1 = makeMatch(1, makeTeam(1, 'H1', 80), makeTeam(2, 'A1',
        50), { sport: 'football' }); // diff=30
660.         const m2 = makeMatch(2, makeTeam(3, 'H2', 80), makeTeam(4, 'A2',
        65), { sport: 'football' }); // diff=15
661.         const m3 = makeMatch(3, makeTeam(5, 'H3', 80), makeTeam(6, 'A3',
        60), { sport: 'football' }); // diff=20
662.         const engine = new RecommendationEngine([m1, m2, m3]);
663.         const user = createUser(1, 'u@test.com', ['football']);
664.
665.         // Act
666.         const recs = engine.generateRecommendations(user);
667.
668.         // Assert – перевіряємо порядок спадання
669.         expect(recs[0].confidence).toBeGreaterThanOrEqual(recs[1].confidenc
        e);
670.         expect(recs[1].confidence).toBeGreaterThanOrEqual(recs[2].confidenc
        e);
671.         expect(recs[0].confidence).toBeCloseTo(0.3, 4);
672.         expect(recs[1].confidence).toBeCloseTo(0.2, 4);
673.         expect(recs[2].confidence).toBeCloseTo(0.15, 4);
674.     });
675.
676.     // GRec-06 – негативний / BVA: preferredSports=[] (межа 0) → виняток
        'no preferred sports'
677.     test('GRec-06 [BVA-] preferredSports=[] (порожній) → кидає помилку
        "no preferred sports"', () => {
678.         // Arrange
679.         const engine = new RecommendationEngine([]);
680.         const user = createUser(1, 'u@test.com', []);
681.
682.         // Act & Assert
683.         expect(() => engine.generateRecommendations(user)).toThrow('no
        preferred sports');
684.     });
685.
686.     // GRec-07 – негативний / EP: user=null → виняток 'non-null object'
687.     test('GRec-07 [EP-] user=null → кидає помилку "non-null object"', ()
        => {
688.         // Arrange
689.         const engine = new RecommendationEngine([]);

```

```

690.
691.         // Act & Assert
692.         expect(() => engine.generateRecommendations(null)).toThrow('non-
null object');
693.     });
694.
695.         // GRec-08 – позитивний / BVA: matches=[] (порожній список) → []
696.         test('GRec-08 [BVA+] matches=[], user з уподобаннями → порожній
масив', () => {
697.             // Arrange
698.             const engine = new RecommendationEngine([]);
699.             const user = createUser(1, 'u@test.com', ['football']);
700.
701.             // Act
702.             const recs = engine.generateRecommendations(user);
703.
704.             // Assert
705.             expect(recs).toHaveLength(0);
706.         });
707.
708.         // GRec-09 – позитивний / EP: case-insensitive зіставлення спорту
709.         test('GRec-09 [EP+] user=["Football"], match.sport="football" → 1
рекомендація (case-insensitive)', () => {
710.             // Arrange
711.             const match = makeMatch(1, makeTeam(1, 'H', 80), makeTeam(2, 'A',
65), { sport: 'football' });
712.             const engine = new RecommendationEngine([match]);
713.             const user = createUser(1, 'u@test.com', ['Football']); // велика
F
714.
715.             // Act
716.             const recs = engine.generateRecommendations(user);
717.
718.             // Assert
719.             expect(recs).toHaveLength(1);
720.         });
721.     });
722.
723.     //

```

```

724.     // 9. createMatch – непокриті негативні гілки (рядки 30, 33, 36)
725.     //

```

```

726.
727.     describe('createMatch', () => {
728.         // CM-01 – негативний / BVA: matchId=0 (межа нуль) → виняток
729.         test('CM-01 [BVA-] matchId=0 → кидає помилку "positive integer"', ()
=> {
730.             // Arrange
731.             const home = makeTeam(1, 'H');
732.             const away = makeTeam(2, 'A');
733.             // Act & Assert

```

```

734.         expect(() => createMatch(0, '2024-01-01', home,
away)).toThrow('positive integer');
735.     });
736.
737.     // CM-02 – негативний / EP: date="" (порожній рядок) → виняток
738.     test('CM-02 [EP-] date="" → кидає помилку "non-empty string", () =>
{
739.         // Arrange
740.         const home = makeTeam(1, 'H');
741.         const away = makeTeam(2, 'A');
742.         // Act & Assert
743.         expect(() => createMatch(1, '', home, away)).toThrow('non-empty
string');
744.     });
745.
746.     // CM-03 – негативний / EP: teamAway=null → виняток
747.     test('CM-03 [EP-] teamAway=null → кидає помилку "required"', () => {
748.         // Arrange
749.         const home = makeTeam(1, 'H');
750.         // Act & Assert
751.         expect(() => createMatch(1, '2024-01-01', home,
null)).toThrow('required');
752.     });
753.
754.     // CM-04 – позитивний / EP: getMatchList повертає копію масиву
755.     test('CM-04 [EP+] getMatchList повертає всі матчі', () => {
756.         // Arrange
757.         const match = makeMatch(1, makeTeam(1, 'H'), makeTeam(2, 'A'));
758.         const svc = new StatsService([match]);
759.         // Act
760.         const list = svc.getMatchList();
761.         // Assert
762.         expect(list).toHaveLength(1);
763.         expect(list[0].matchId).toBe(1);
764.     });
765. });
766.
767. //

```

```

768. // 10. createUser – непокриті негативні гілки (рядки 53, 56)
769. //

```

```

770.
771. describe('createUser', () => {
772.     // CU-01 – негативний / BVA: userId=0 (межа нуль) → виняток
773.     test('CU-01 [BVA-] userId=0 → кидає помилку "positive integer"', ()
=> {
774.         // Act & Assert
775.         expect(() => createUser(0, 'u@test.com')).toThrow('positive
integer');
776.     });
777.
778.     // CU-02 – негативний / EP: email без @ → виняток

```

```

779.     test('CU-02 [EP-] email="notanemail" → кидає помилку "valid email"',
    () => {
780.         // Act & Assert
781.         expect(() => createUser(1, 'notanemail')).toThrow('valid email');
782.     });
783. });
784.
785. //

```

```

786. // 11. RecommendationEngine – непокриті гілки (рядки 176, 187)
787. //

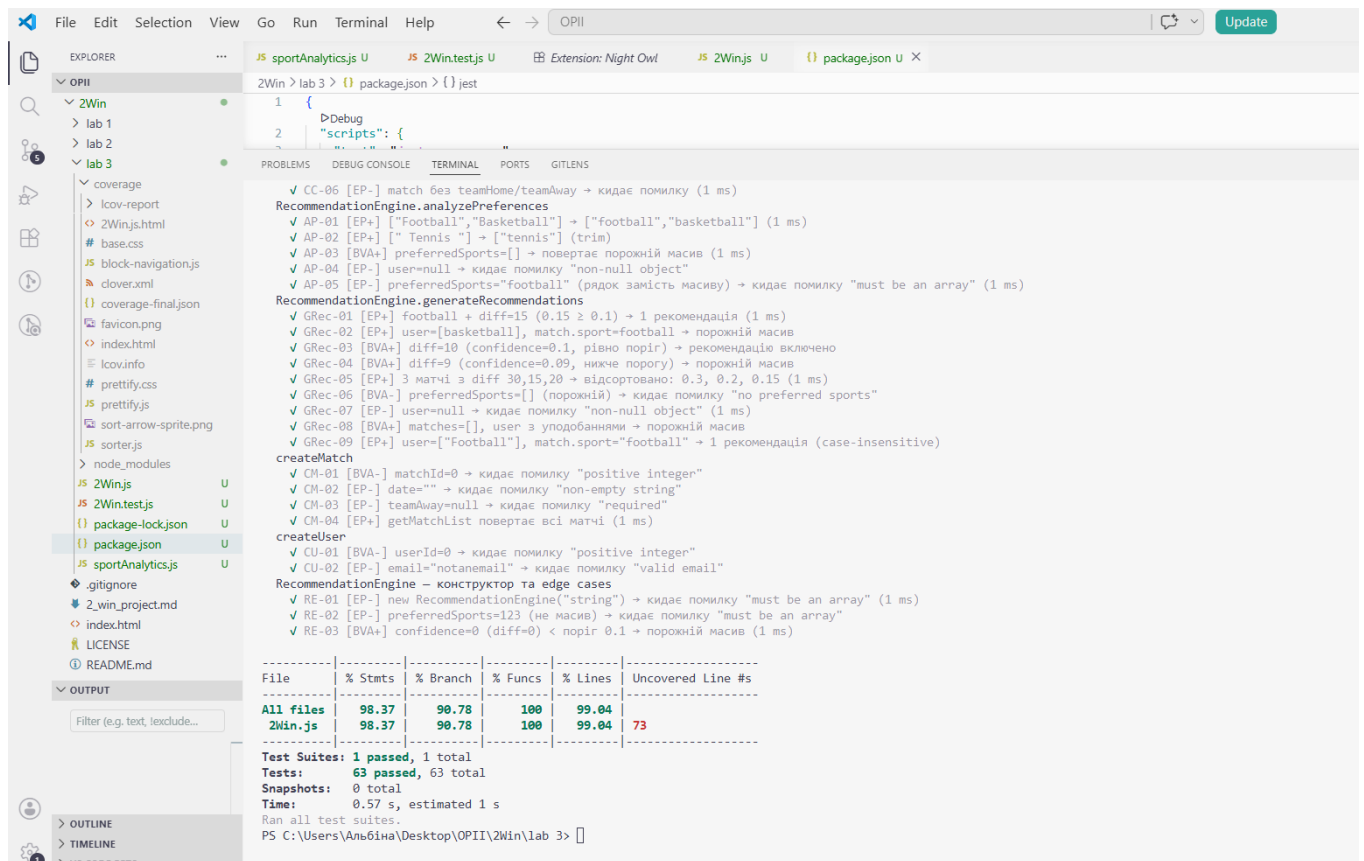
```

```

788.
789. describe('RecommendationEngine – конструктор та edge cases', () => {
790.     // RE-01 – негативний / EP: конструктор отримує не масив → виняток
791.     test('RE-01 [EP-] new RecommendationEngine("string") → кидає помилку
    "must be an array"', () => {
792.         // Act & Assert
793.         expect(() => new RecommendationEngine('not-array')).toThrow('must
    be an array');
794.     });
795.
796.     // RE-02 – негативний / EP: generateRecommendations – preferredSports
    не масив → виняток
797.     test('RE-02 [EP-] preferredSports=123 (не масив) → кидає помилку
    "must be an array"', () => {
798.         // Arrange
799.         const engine = new RecommendationEngine([]);
800.         const user = { userId: 1, preferredSports: 123 };
801.         // Act & Assert
802.         expect(() => engine.generateRecommendations(user)).toThrow('must be
    an array');
803.     });
804.
805.     // RE-03 – позитивний / BVA: confidence=0 (diff=0) → нижче порогу,
    рекомендацій немає
806.     test('RE-03 [BVA+] confidence=0 (diff=0) < поріг 0.1 → порожній
    масив', () => {
807.         // Arrange
808.         const match = makeMatch(1, makeTeam(1, 'H', 50), makeTeam(2, 'A',
    50), { sport: 'football' });
809.         const engine = new RecommendationEngine([match]);
810.         const user = createUser(1, 'u@test.com', ['football']);
811.         // Act
812.         const recs = engine.generateRecommendations(user);
813.         // Assert
814.         expect(recs).toHaveLength(0);
815.     });
816. });

```

Скріншот та HTML-звіт покриття коду з відображенням відсотку line coverage:



HTML:

All files									
98.37%	Statements	121/123	90.78%	Branches	128/141	100%	Functions	27/27	99.04% Lines 104/105
Press <i>n</i> or <i>j</i> to go to the next uncovered block, <i>b</i> , <i>p</i> or <i>k</i> for the previous block.									
Filter:									
File	Statements	Branches	Functions	Lines					
2Win.js		98.37%	121/123	90.78%	128/141	100%	27/27	99.04%	104/105

Code coverage generated by [istanbul](#) at 2026-06-02T12:25:47.771Z

Git-репозиторій:

Mykhailo-Berkuta/2Win at feature/Zhakun

Висновок:

У ході виконання лабораторної роботи було розроблено набір із 63 модульних тестів для модуля 2Win.js, що реалізує функціональність сервісу статистики (StatsService) та рушія рекомендацій (RecommendationEngine). За результатами запуску тестового набору отримано такі метрики покриття коду:

- покриття рядків (Line Coverage): 99.04%
- перевищує встановлений поріг у 80% майже на 20 відсоткових пунктів;
- покриття гілок (Branch Coverage): 90.78%
- усі основні умовні конструкції перевірено;
- покриття функцій (Function Coverage): 100%
- кожна функція та метод були викликані під час тестування.

Єдиний непокритий рядок (73) належить конструктору класу StatsService і відповідає перевірці вхідного параметра matches на тип масиву – ця гілка не була активована жодним тестом через особливості ініціалізації об'єктів у тестовому середовищі.

Виявлені проблеми

У процесі проєктування та виконання тестів було виявлено низку недоліків:

По-перше, функції createMatch та createUser спочатку не мали жодного тестового покриття для негативних сценаріїв, що було виявлено лише після аналізу звіту покриття. Це свідчить про важливість ітеративного підходу до написання тестів.

По-друге, метод filterStats з критерієм draw повертав більше матчів, ніж очікувалось, через те що матчі без явно заданого рахунку мають значення 0:0 за замовчуванням і також потрапляють під умову нічиєї. Це виявило потенційну логічну проблему в самому модулі.

По-третє, метод _estimateOdds залишився частково непокритим по гілках, оскільки виклик з confidence=0 не генерує рекомендацію через фільтрацію за порогом – внутрішній метод недоступний для прямого тестування ззовні.

Шляхи поліпшення

Для підвищення якості тестового набору та самого модуля можна запропонувати такі вдосконалення:

- додати тест для конструктора StatsService з невалідним аргументом (не масив), що дозволить досягти 100% покриття рядків;
- винести _estimateOdds у публічний метод або додати інтеграційний тест, який генерує рекомендацію з confidence=1.0 для перевірки мінімального коефіцієнта 1.01;

- для методу `filterStats` уточнити логіку: розрізняти матчі, що ще не зіграні (рахунок 0:0 за замовчуванням), від справжніх нічий, наприклад додавши поле `status: 'finished' | 'upcoming'`;
- підключити CI/CD пайплайн (GitHub Actions) з прапорцем `--coverageThreshold`, що автоматично блокуватиме злиття гілок із покриттям нижче 80%, забезпечуючи підтримку якості на рівні репозиторію.